CASE STUDY REPORT

Practo: Cloud Architecture on AWS

Problems faced, how the architecture evolved, and the solutions built

Contents

1. Introduction
2. Problems and Bottlenecks
3. Solutions Practo Built
4. Architecture Overview
5. Future Plans
6. Conclusion
7. References

1. Introduction

Practo is a healthcare platform based in Bangalore, India [1]. It serves two groups of users. Patients use it to find the right doctor and book an appointment. Providers (doctors, clinics and hospitals) use Practo's software, such as Practo Ray, to run their practices [2].

The Practo engineer says millions of patients use the platform every month, and that its software for doctors is used in 18 different countries [5]. In May 2016, Bloomberg reported 60 million users, 200,000 doctors, 10,000 hospitals and more than 40 million appointments a year [1]. The founder has quoted a similar number of appointments [3]. An AWS India executive has also named Practo as an example AWS customer [4]. These figures are older and should not be read as current counts.

This report explains the problems Practo faced, how its architecture on AWS changed over time, the solutions it built, and the main numbers behind the results.

2. Problems and Bottlenecks

Practo started as a single large application (a monolith) running on AWS Auto Scaling. As the business grew, the team split it into many smaller services. This brought four main problems [5], [6].

2.1 Dependence on the infrastructure team

As the number of services grew, developers had to wait for the infrastructure team to set up each new service. This slowed down development and created a dependency between the two teams [5].

2.2 High cost from many Auto Scaling groups

Each service ran in its own Auto Scaling group. Many groups meant a lot of spare capacity that was paid for but not fully used [5].

2.3 Mixing real-time requests and background jobs

User-facing (transactional) requests must be answered quickly. Background (asynchronous) jobs can wait. Practo needed to keep these two apart so that background jobs did not need always-on servers [5].

2.4 Database performance

Practo's Amazon RDS databases ran MySQL 5.1. Queries created automatically by the application were slow, and disk I/O and temporary-table limits hurt performance under heavy load. This slowed searches and bookings at peak hours, while RDS costs kept rising [6].

3. Solutions Practo Built

The table gives a quick view. The sub-sections explain each solution in more detail.

Problem	Solution	Result
Waiting on infrastructure team	Dockerfile per service, CI/CD pipeline and an in-house DSL	Developers deploy on their own
High cost, many Auto Scaling groups	One Kubernetes cluster with larger instances	More than 40% cost saving
Mixed real-time and background work	SQS queues with workers on spot instances	Cheaper and faster job processing
Slow database	Provisioned IOPS, upgrades query, indexing and schema tuning	Better speed at peak hours

3.1 Self-service for developers

Every service has a Dockerfile in its own codebase and an automatic build on every push. The DSL lets a developer ask for supporting infrastructure in a simple way. Together these removed the waiting time between development and infrastructure teams [5]. The engineer says Practo makes around 2500 releases each month across the 80 services without such dependency.

3.2 One shared Kubernetes cluster

Moving from many Auto Scaling groups to one Kubernetes cluster with larger instances allowed many services to share the same servers resulting in better utilisation. The engineer reports a saving of more than 40% in cost [5].

3.3 Cheaper background processing

Putting jobs in SQS separates background work from user requests. Workers run on spot instances, which cost less. Prometheus metrics and the Worker Pod Autoscaler let the workers grow and shrink with the number of jobs waiting. The engineer says this “worked wonders” in how fast jobs are scaled and processed [5].

3.5 Database tuning

A case study by Mydbops describes the work on RDS [6]:

- Provisioned IOPS to improve disk performance.
- Upgrades from MySQL 5.1 to 5.5 and then 5.6.
- Rewritten slow queries, cleaner schemas and better indexes.
- Online schema changes built into the deployment pipeline, so changes do not need downtime.

4. Architecture Overview

The description below combines the engineer’s walk-through [5] with Practo’s public code [7], [8]. It is our reading of those sources, not an official diagram. No AWS regions are published.

4.1 Request path

A user request first reaches an Auto Scaling group behind a load balancer. The load balancer sends it to NGINX router machines. These pass the request to a second load balancer in front of the Kubernetes cluster, where the services run. The cluster is created and managed with kops on AWS [5].

4.2 Developer workflow

Each service lives in one codebase with its own Dockerfile. When a developer pushes code, the CI/CD pipeline picks up the Dockerfile and builds an image. The same image is used on a local machine, in staging and in production [5].

Practo also built a small in-house domain-specific language (DSL). Developers use it to declare the extra infrastructure a service needs, such as RDS, Memcached and Redis, without waiting for the infrastructure team [5].

4.3 Background jobs

When a service needs to run a background job, it places the job in an Amazon SQS queue. Worker pods listen to the queue and process the jobs. These workers run on cheaper spot instances because they do not need servers running all the time [5].

Practo built a Worker Pod Autoscaler to scale the workers [7]. Queue metrics from SQS are pushed into Prometheus, and the autoscaler reads prometheus metrics to add or remove workers [5]. Its documentation says that queue length alone can mislead, because fast workers keep the backlog near zero. So it also looks at message activity and processing time, and it allows a minimum of zero workers [7].

4.4 Logging, monitoring and data

- **Load balancer logs:** stored in Amazon S3 [5].
- **Request logs:** sent to Graylog, where Practo also creates alarms and alerts [5].
- **Errors and exceptions:** sent to Sentry, with monitoring and alerts so problems are found and fixed early [5].
- **Data pipeline:** Tipoca Stream is described as a near-real-time pipeline from RDS to Redshift, with masking and reloading support [8].

4.5 Database layer

Practo uses Amazon RDS with MySQL, tuned as described in Section 3.5.

4.6 Architecture diagram

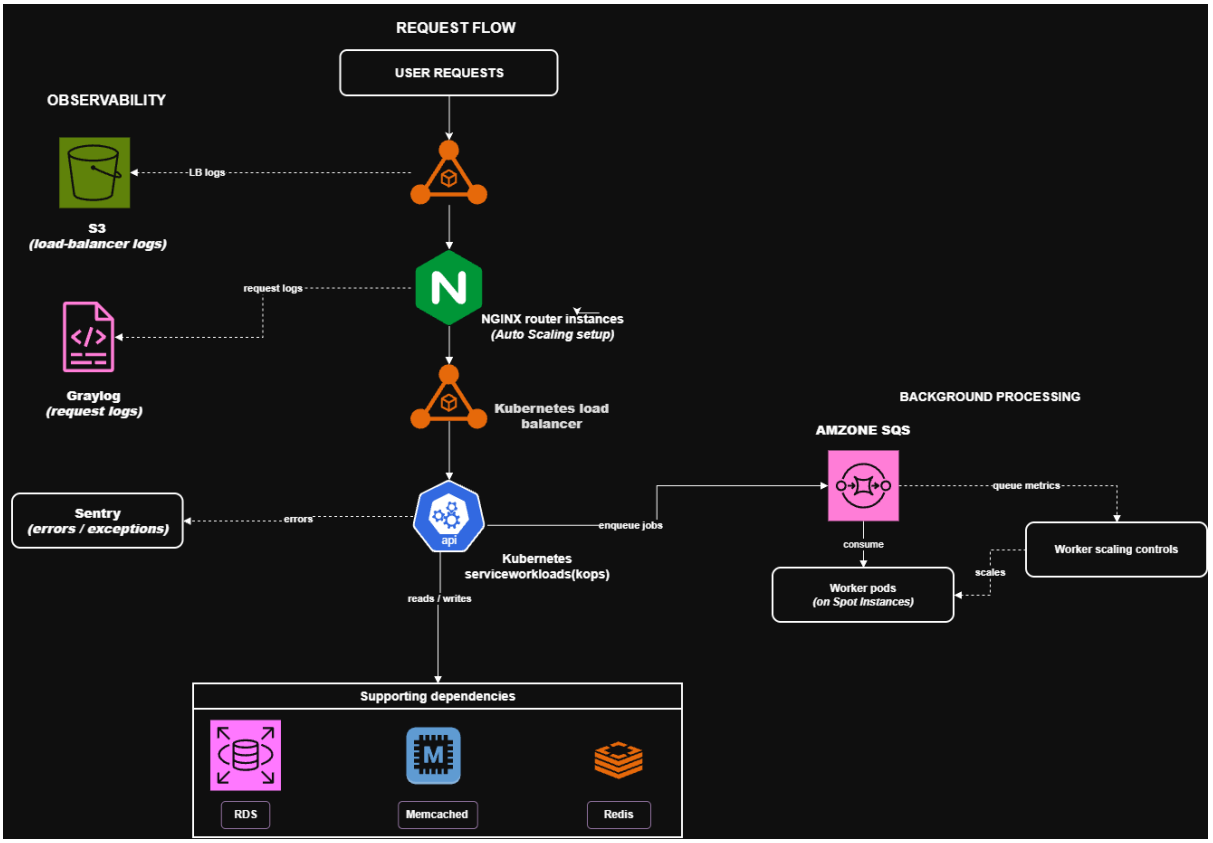


Figure 1: Practo architecture on AWS

7. Future Plans

The engineer names two next steps [5]:

- **Service mesh:** to better manage how requests move between services and how load is shared.
- **Amazon EKS:** to hand over Kubernetes management to AWS, so Practo's team can spend more time on new features.

9. Conclusion

Practo's main challenges were slow delivery caused by team dependencies, high infrastructure cost, mixed workloads and a busy database. It answered with a few clear ideas: let developers help themselves with Docker, CI/CD and a DSL; run services together on one Kubernetes cluster; move background work to SQS with spot-based workers that scale on queue metrics; and tune the database step by step.

The reported result is a platform of close to 80 services, more than 40% lower cost, and a simple path for developers to release changes. Practo plans to continue with a service mesh and Amazon EKS.

References

[1] Bloomberg, “India’s Practo plans to take digital healthcare success abroad,” May 27, 2016. https://www.bloomberg.com/news/articles/2016-05-27/india-s-practo-plans-to-take-digital-health-care-success-abroad

[2] TechCrunch, “Indian health search portal Practo lands $90M led by Tencent for global expansion,” Aug. 6, 2015. https://techcrunch.com/2015/08/06/indian-health-search-portal-practo-lands-90m-led-by-tencent-for-global-expansion

[3] DQ India, “Practo is the world’s largest healthcare appointment booking platform: Shashank ND, Founder, Practo.” https://www.dqindia.com/practo-is-the-worlds-largest-healthcare-appointment-booking-platform-shashank-nd-founder-practo/

[4] CXOToday, “How AWS is driving India’s cloud growth story.” https://cxotoday.com/cloud/aws-and-cloud-growth-to-accelerate/

[5] Amazon Web Services, “This Is My Architecture: Practo,” YouTube video. https://youtu.be/YgABcxC4pwQ

[6] Mydbops, “Practo AWS RDS database optimization case study.” https://www.mydbops.com/case-studies/practo-database-optimization-aws-rds (accessed Oct. 21, 2026).

[7] Practo, “k8s-worker-pod-autoscaler,” GitHub. https://github.com/practo/k8s-worker-pod-autoscaler (accessed Oct. 21, 2026).

[8] Practo, GitHub organization (Tipoca Stream). https://github.com/practo (accessed Oct. 21, 2026).